Lab Document: Understanding Node Affinity, Pod Anti-Affinity, and Node Labeling in Kubernetes

---

Overview:

In this lab, we will walk through the concepts of **Node Affinity**, **Pod Anti-Affinity**, and **Node Labeling** in Kubernetes. We'll explore how to control where pods are scheduled within a Kubernetes cluster, and how to use labels to manage pod placement using affinity and anti-affinity rules.

---

1. Initial Setup:

Before diving into the Kubernetes concepts, let's start by creating a basic **Deployment** and label the nodes in your Kubernetes cluster. This lab assumes you already have a working Kubernetes cluster with at least two nodes.

1.1 Deployment YAML (Initial Deployment Script):

Here’s the initial script for creating a simple Deployment that will run 3 replicas of a container on any available node.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
```

```
  metadata:
    labels:
      app: my-app
  spec:
    containers:
    - name: ec-app
      image: akiranreddy/ec2
      ports:
      - containerPort: 80
```

You can create this deployment by saving the file as deployment.yaml and running the following command:

kubectl apply -f deployment.yaml

1.2 Labeling the Nodes:

To simulate a **Node Affinity** scenario, we need to label the nodes.

Assume your nodes have names like node1, node2, etc.

Run the following commands to label your nodes (replace with actual node names):

kubectl label nodes <node-name> nodeName=node1
kubectl label nodes <node-name> nodeName=node2

This step will label your nodes with nodeName=node1 and nodeName=node2.

2. Node Affinity

2.1 What is Node Affinity?

Node Affinity allows you to control where your pods are scheduled based on the labels of the nodes. You can specify hard or soft requirements for node selection using nodeAffinity.

2.2 Modifying the Deployment to Run Pods Only on node1:

In the previous script, we will modify the **affinity** section to ensure that our pods **only run on node1**.

Updated YAML with Node Affinity:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - key: nodeName
                operator: In
                values:
                - node1
      containers:
      - name: ec-app
        image: akiranreddy/ec2
        ports:
```

```
    - containerPort: 80
```

Explanation:

- nodeAffinity ensures that the pods can only be scheduled on nodes labeled with nodeName=node1.
- requiredDuringSchedulingIgnoredDuringExecution specifies that the rule must be satisfied at scheduling time.
- operator: In means the pod can only run on nodes that have nodeName=node1.

To apply the updated deployment:

kubectl apply -f updated-deployment.yaml

2.3 Verify Pod Placement:

After applying the updated deployment, check the placement of your pods:

kubectl get pods -o wide

You should see that all pods are scheduled on **node1**.

---

3. Pod Anti-Affinity

3.1 What is Pod Anti-Affinity?

Pod Anti-Affinity prevents a pod from being scheduled on the same node as other pods with a specific label. This helps to distribute the workload across multiple nodes and provides **high availability**.

3.2 Modifying the Deployment to Avoid Pods on the Same Node:

Now, let's modify the Deployment to include **Pod Anti-Affinity** rules. We'll ensure that pods of my-app are **not scheduled on the same node**.

Updated YAML with Pod Anti-Affinity:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - my-app
            topologyKey: kubernetes.io/hostname
      containers:
      - name: ec-app
        image: akiranreddy/ec2
        ports:
```

```
    - containerPort: 80
```

Explanation:

- podAntiAffinity ensures that the pod will **not be scheduled on the same node** as another pod with label app=my-app.
- topologyKey: kubernetes.io/hostname means that the anti-affinity rule is enforced at the **node level**. This ensures that pods are spread across different nodes.

3.3 Verify Pod Distribution:

After applying the updated deployment, check where the pods are placed:

kubectl get pods -o wide

You should see that the pods are spread across different nodes (e.g., node2 and node3).

---

4. Combining Node Affinity and Pod Anti-Affinity

In real-world scenarios, you often need to combine both **Node Affinity** and **Pod Anti-Affinity**.

For example, you might want to:

- Run pods only on node1 (Node Affinity).
- Avoid scheduling more than one pod of the same app on the same node (Pod Anti-Affinity).

Combined YAML Example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - key: nodeName
                operator: In
                values:
                - node1
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - my-app
```

```
      topologyKey: kubernetes.io/hostname
  containers:
  - name: ec-app
    image: akiranreddy/ec2
    ports:
    - containerPort: 80
```

Explanation:

- This configuration combines **Node Affinity** to run the pods only on node1 and **Pod Anti-Affinity** to avoid running multiple pods of my-app on the same node.

- The **anti-affinity** ensures that even if you try to run 3 replicas on node1, the pods will be distributed across different nodes (if available).

4.1 Apply the Combined Deployment:

kubectl apply -f combined-deployment.yaml

---

5. Conclusion:

- **Node Affinity**: Restricts where the pod can be scheduled based on node labels.

- **Pod Anti-Affinity**: Ensures that your pods are not scheduled on the same node as other pods with specific labels.

- By combining both, you can fine-tune pod placement in your Kubernetes cluster to ensure **high availability** and **optimized resource utilization**.

Next Steps:

- You can experiment with **soft affinity rules** (preferredDuringSchedulingIgnoredDuringExecution) to create **less strict placement rules**.

- Learn how to use **taints and tolerations** to fine-tune pod scheduling further.